Builder NPC Plugin for Minecraft Server

CS39440 Major Project Report

Author: Hannah Frick (haf31)

Supervisor: Dr. Jim Finnis (jcf12)

30th April 2026

Version 1.0 (Release)

This report is submitted as partial fulfilment of a BSc degree in Computer Science (G400)



Department of Computer Science, Aberystwyth University,
Aberystwyth, Ceredigion, SY23 3DB, Wales, UK

Declaration of originality

I confirm that:

- This submission is my own work, except where clearly indicated.
- I understand that there are severe penalties for Unacceptable Academic Practice, which can lead to loss of marks or even the withholding of a degree.
- I have read the regulations on Unacceptable Academic Practice from the University's Academic Registry (AR) and the relevant sections of the current Student Handbook of the Department of Computer Science.
- I have read the document Generative AI and the Major and Minor Projects, which is available in the Blackboard module.
- In submitting this work, I understand and agree to abide by the University's regulations governing these issues.

Name Hannah Frick

Date 30/04/2026

Consent to share this work

By including my name below, I hereby agree to this project's report and technical work being made available to other students and academic staff of the Aberystwyth Computer Science Department.

Name Hannah Frick

Date 30/04/2026

Acknowledgements

I would like to thank my best friend Megan and my lovely boyfriend Rhys for being the most supportive people throughout my project and University years. I would also like to thank my supervisor Dr. Jim Finnis for guiding me through the project and inspiring me to do better with my education as a whole. Finally, I would like to thank my parents for supporting me through everything and being there for me when I need them even with being across the pond.

AI Declaration

Generative AI was used for spell check throughout this report.

Abstract

This project was about designing and developing a Minecraft NPC builder plugin which allows players to select a building schematic through chat commands. The chat commands tell the NPC which schematic the user wants so it can then find a flat bit of land to build the chosen schematic. The plugin uses the Citizens 2 API to create and control the NPC, allowing it to have custom behaviors such as walking around the world and acting as the builder. The plugin also uses the Bukkit API to interact with the Minecraft server, such as detecting player chat events, seeing world locations, checking block types, and scheduling tasks. Together these APIs allow the plugin to connect player input, NPC movement, land detection, and automatic building into one working system.

Contents

1. BACKGROUND, ANALYSIS & PROCESS 10

1.1. Background 10
1.1.1. Minecraft and Server Plugins 10
1.1.2. Paper vs Spigot 10
1.1.3. Bukkit API 10
1.1.4. Citizens2 API 10
1.1.5. Project Overview 11

1.2. Analysis 11
1.2.1. Problem 11
1.2.2. Project Aim 12
1.2.3. Requirements 12
1.2.4. Challenges 13
1.2.5. Alternative Approaches 13

1.3. Process 14
1.3.1. Methodology 14
1.3.2. Planning and Task Management 15
1.3.3. Version Control 15

1.4. Status of work 16
1.4.1. Completed Work 16
1.4.2. Current Capabilities 17
1.4.3. Unfinished 17
1.4.4. Known Issues 17
1.4.5. Planned Future Work 18

2. DESIGN 19

2.1. Design Overview 19
2.1.1. Design Goals 19
2.1.2. Scope 19

2.2. System Architecture 19
2.2.1. Architecture Diagram 20
2.2.2. Component Responsibilities 20
2.2.3. Data Flow 21
2.2.4. State Management 22

2.3. Schematics 23
2.3.1. Schematic Implementation 23
2.3.2. Build Origin 24
2.3.3. Directional Blocks 24

2.4. Search 24
2.4.1. Valid Build Site 24
2.4.2. Search Space and Parameters 24
2.4.3. Grid Search 24
2.4.4. Validation 24
2.4.5. Failure 24

2.5. NPC Behaviour 25

2.5.1. Citizen Traits ... 25
2.5.2. Navigation ... 25
2.5.3. Scheduler ... 25
2.5.4. Commands ... 25

3. IMPLEMENTATION ... 26

3.1. Implementation Overview ... 26
3.1.1. Development Environment and Setup ... 26
3.1.2. Project Structure ... 27

3.2. Plugin Entry and Event Handling ... 27
3.2.1. Plugin Configuration ... 27
3.2.2. Plugin Lifecycle ... 27
3.2.3. Chat Event Handling ... 28

3.3. Citizens NPC Integration ... 28
3.3.1. NPC Creation and Reuse ... 28
3.3.2. Navigation Setup ... 28
3.3.3. Trait Attachment ... 29

3.4. Schematic Loading ... 29
3.4.1. File Location ... 29
3.4.2. Schematic Format ... 29
3.4.3. Directional Blocks ... 29

3.5. Flat Land Search Implementation ... 29
3.5.1. Candidate ... 29
3.5.2. Algorithm Used ... 30
3.5.3. Validation ... 30
3.5.4. Return Build location ... 30

3.6. Scheduling ... 30
3.6.1. Bukkit Scheduler ... 30
3.6.2. Tick timing ... 30

3.7. Building Implementation ... 31
3.7.1. Build Trigger ... 31
3.7.2. Block Placement ... 31

3.8. Problems Encountered ... 31
3.8.1. NPC Confused ... 32
3.8.2. Search/Build Command Coupling ... 32

4. TESTING ... 33

4.1. Functional Testing ... 33

4.2. User Testing ... 34
4.2.1. Experienced ... 34
4.2.2. Non-Experienced ... 36

5. CRITICAL EVALUATION ... 39

5.1. Strengths ... 39
5.1.1. Technical Strengths ... 39
5.1.2. Usability Strengths ... 39

5.1.3. Process Strengths 39

5.2. Weaknesses 40
5.2.1. Technical Limitations 40
5.2.2. Reliability Limitations 40
5.2.3. Usability Limitations 40

5.3. Improvements if Restarting 40
5.3.1. Design Improvements 40
5.3.2. Implementation Improvements 41
5.3.3. Testing Improvements 41

5.4. Future Work 41
5.4.1. Schematics 41
5.4.2. Directional Blocks 41
5.4.3. Faster Search 41

5.5. Conclusion 42
5.5.1. Summary 42
5.5.2. Final Reflection 42

6. REFERENCES 43

7. APPENDICES 44

A. Use of Third-Party Code and Libraries 45

1. Background, Analysis & Process

1.1. Background

This introduces the technologies needed to understand the project. It explains Minecraft server plugins, the difference between Paper and Spigot, Bukkit API, and Citizens2 API. These provide the server environment, API access, and NPC control needed for the builder plugin

1.1.1. Minecraft and Server Plugins

Minecraft is a sandbox game that has no goals meaning that players can be as creative as they wish. This includes doing tasks like building, mining, exploring, or interacting with the many creatures in the game [1].

Minecraft servers allow multiple players to join the same world and play together as a group, or the user can play by themselves and enhance the game using mods or server plugins.
Server plugins are add ons that can change how a Minecraft server works and functions. They allow the user to add new commands, NPCs, features, or new tools. For this project, a server plugin was created to add an NPC builder into Minecraft allowing the player to type chat commands to choose a building schematic then watch the NPC search for land to place the building on.

1.1.2. Paper vs Spigot

Spigot and Paper are both Minecraft server creation software's that support plugins. Spigot is one of the most commonly used software's for Minecraft plugins and supports the Bukkit API meaning that developers can write Java code that interacts with the world, players, blocks, and events [2].

Paper is based on Spigot but was built to improve performance. This means Paper can run more efficiently than Spigot especially on servers with multiple plugins. Paper also uses the Bukkit API and most plugins made for Spigot can run on Paper as well [3].
For the project, Paper was used for the server creation software. It was chosen due to it providing better performance and improvements while also still utilizing everything Spigot had to offer, this made it suitable for running the NPC builder plugin in Minecraft.

1.1.3. Bukkit API

The Bukkit API is a Java programming interface used to help create Minecraft server plugins. An API allows a developer to use existing classes and methods instead of writing everything from scratch. It allows plugins to interact with different parts of a server such as words, blocks, events, and scheduled tasks [4], [5].
In this project, Bukkit provides the connection between the code and the Minecraft server. It was used for the main plugin functionality by registering the plugin when the server starts, listen for player chat commands, find world locations, validate the flat land, and build the selected schematic.

1.1.4. Citizens2 API

Citizens2 is a Minecraft plugin that allows servers to create and control NPCs. Citizens" provides an API that developers can use to create NPCs, give them names, attach traits to them, and control how they move and interact with the world [6].

For the project, Citizens2 was used to create the builder NPC named Jeff. The API allowed the plugin to create Jeff if he didn't already exist or reuse him if he was already recreated without having duplicates. Citizens2 also allowed for traits, which are custom behaviours that are attached to the NPC. A custom builder trait was attached to Jeff, this controlled the main behaviour of him like storing the schematics, starting the flat land search, moving towards the flat land, and building the schematic after the player typed build [7] [8].

1.1.5. Project Overview

This project is a Minecraft NPC builder plugin that allows a player to choose a building schematic and have the NPC find a suitable place to build it.

The player interacts with the plugin through chat commands. When the player types of schematics, the plugin then returns a list of all schematics to the user. When the player types the name of a schematic, the plugin loads the matching text file and calculates the width and length of the building. It then searches the world for a flat area that is large enough for the structure. When a suitable location is found, the NPC moves towards the location using the Citizens2 navigation system, then when the NPC reaches the location, the plugin stores the coordinates and returns it back to the player. The player can then type build, which has the plugin read the schematic file and place the building into the world.

The plugin uses several classes to separate the different parts of the system. The main plugin class handles server start up, chat messages, and creating the NPC. The builder trait class stores the schematic and controls when the search or build process should happen. The flat land class checks possible locations and makes sure the chosen area is level. The schematic class stores the dimensions of the building so the plugin knows the width and length to search for.

1.2. Analysis

This section analyses the problem that the project is trying to solve and explains the aim, requirements, challenges, and possible approaches. It identifies why the NPC cannot just build anywhere it wants in the world and why the plugin needs to validate the land before placing the building. It also explains design decisions made such as using text based schematics, calculating the width and length of the schematic, and using a grid based search to find flat land.

1.2.1. Problem

The problem this project is trying to solve is how to create a Minecraft server plugin where an NPC can automatically build structures from stored blueprints. In normal Minecraft gameplay, players must choose a location, check if the land is suitable, and manually place the blocks themselves. This project aims to make the players life easier by providing an NPC with the ability to read a building schematic, search for the right bit of land, and build the structure.

A major part of the problem is that Minecraft worlds are unpredictable and therefore uneven. The land can have hills, caves, water, trees, lava, and many other features making it unsuitable to build on. If the plugin places a building without checking the terrain first, the building could overlap with hills or be partly placed over water and lava. Which is why the NPC must be able to find a flat area large enough for the selected building.

Another issue is that different buildings require different amounts of space. A 5x5 land search is only suitable for smaller buildings and does not work when larger buildings are introduced. To solve this, the plugin needs to be able to calculate the required width and length from the selected schematic then search for land based on the chosen structure.

1.2.2. Project Aim

The aim of the project is to develop a Minecraft Paper server plugin, Using the Bukkit and Citizens2 APIs that allows an NPC to find suitable flat land and build the players selected building from schematic files.

It aims to combine several technical areas such as player input through chat commands, Citizens NPC navigation, schematic file loading, terrain validation, and Bukkit block placement. A goal is for the land search to be based on the schematic's width and length, so larger buildings require larger flat areas and smaller buildings can be put in smaller places.

1.2.3. Requirements

1.2.3.1 Functional Requirements

Table 01: *Functional Requirements of the Project*

ID	Requirement
FR1	Must run as a paper Minecraft server plugin using Bukkit API and written in Java
FR2	Must use the Citizens2 plugin
FR3	The plugin must create or reuse a Citizens NPC
FR4	The NPC needs a custom trait that controls the building behaviour
FR5	The plugin must allow the player to see the list of schematics after typing schematics
FR6	The plugin must allow the player to type the name of the selected schematic
FR7	The plugin must store the schematics in a format that can be read
FR8	The plugin must calculate the width and length based on the selected schematic
FR9	The NPC needs to move to the flat land using Citizens navigation
FR10	The plugin must construct the selected building after the player types build
FR11	The plugin should return the coordinates of the flat land found to the player
FR12	The plugin should allow multiple schematic files

1.2.3.2 Non-functional Requirements

Table 02: *Non-functional Requirements of the Project*

ID	Requirement
NFR1	Show a building animation of the NPC placing the blocks one by one

NFR2	Schematic format should be easy to edit and extend
NFR3	Should be understandable and demonstrable through simple player commands

1.2.4. Challenges

The project presented several technical challenges. The first challenge was learning how to set up and run a Minecraft server capable of supporting the Bukkit and Citizens2 APIs. This involved installing Paper, adding the Citizens2 plugin, and testing that it could load correctly onto the server.

A second challenge was learning the Bukkit API. The plugin needed to be able to listen to player chat events, access world locations, check block, place blocks, And schedule repeated tasks. These features are o=in different parts of the API, so making a tester plugin was needed before attempting to make the final thing.

A third challenge was learning the Citizens2 API. Citizens2 was required to create an NPC and attach custom behaviour with a trait. This was important for the NPC to move through the world and have builder behaviour such as the selected schematic and found build location.

The main technical challenge was the land search. Minecraft terrain is very uneven, and the NPC cannot place a building unless the selected area is large and flat enough The search needed to check the required width and length of the schematic and reject uneven terrains such as water, lava, and different ground height.

Another challenge was storing and reading schematic blueprints. A text based schematic format was chosen because it was simple to write and add on to. Directional blocks though such as stairs, doors, and windows were difficult because they require block data such as the direction its facing and block material type.

1.2.5. Alternative Approaches

Several approaches were considered during development. One option was to hardcode the schematics. While this would've been simple for one building, it would've got complicated adding more buildings as each one would require code changes. A text based format was used instead because new buildings could be easily added just by creating a new text file.

A second alternative was to use a fixed land search size such as always searching for a 5x5 area. This was suitable for the testing building but was very limiting when new buildings were introduced. The chosen approach was to calculate the width and length from the selected schematic, meaning that the land search is based on the building selected.

Different search algorithms were also looked at, however the best suitable search for me was a grid/ring search. It checks nearby areas first and is simple to explain and implement. Citizens2 then handles the actual pathfinding once a target has been found.

Finally, the building process could have been animated so the NPC could place the blocks in real time. However, due to the project timeline, the implemented version just places the building after the site has been found.

1.3. Process

This section explains the process used to manage and develop the project. It describes the iterative methodology used, how GitLab's Kanban issue board supported planning, and how version control was used during development. It also outlines the main milestones completed during the project and how the work progressed from server start up to a working NPC builder plugin.

1.3.1. Methodology

This project followed an iterative development methodology supported by *GitLab*'s built in Kanban issue board [11] look at ***Figure 01*** for an example. This was chosen because the project involved learning and implementing several different technologies, including Paper, Bukkit API, and Citizens2 API. It was difficult to know what issues could come up at the beginning of the project because many didn't even become clear until tested in Minecraft multiple times.

The project was developed in small stages. Each feature was implemented, tested in the server, then improved on, before moving onto the next feature. For example, the first test was making sure that Citizens2 was able to load in an NPC. After this, the custom trait was added, then flat land search, and finally schematic buildings.

This methodology suited the project because the NPC needed to be constantly tested in game. Problems such as the NPC moving to invalid locations, not finding suitable land, and directional blocks like stairs, would not of been able to be improved upon without running the plugin in Minecraft. Iterative development made is possible to identify these issues, adjust the code, and retest.

Kanban was used to help organise the project into smaller tasks. The tasks could be placed into different stages such as "Open", "In progress", and "Done". The Kanban board that was utilized is built into Gitlab and made it easy to have all of the project in one place, however, it was difficult to remember to move the tasks not just look at them. It did make it easier though to know which tasks needed to be a priority by seeing them listed out.

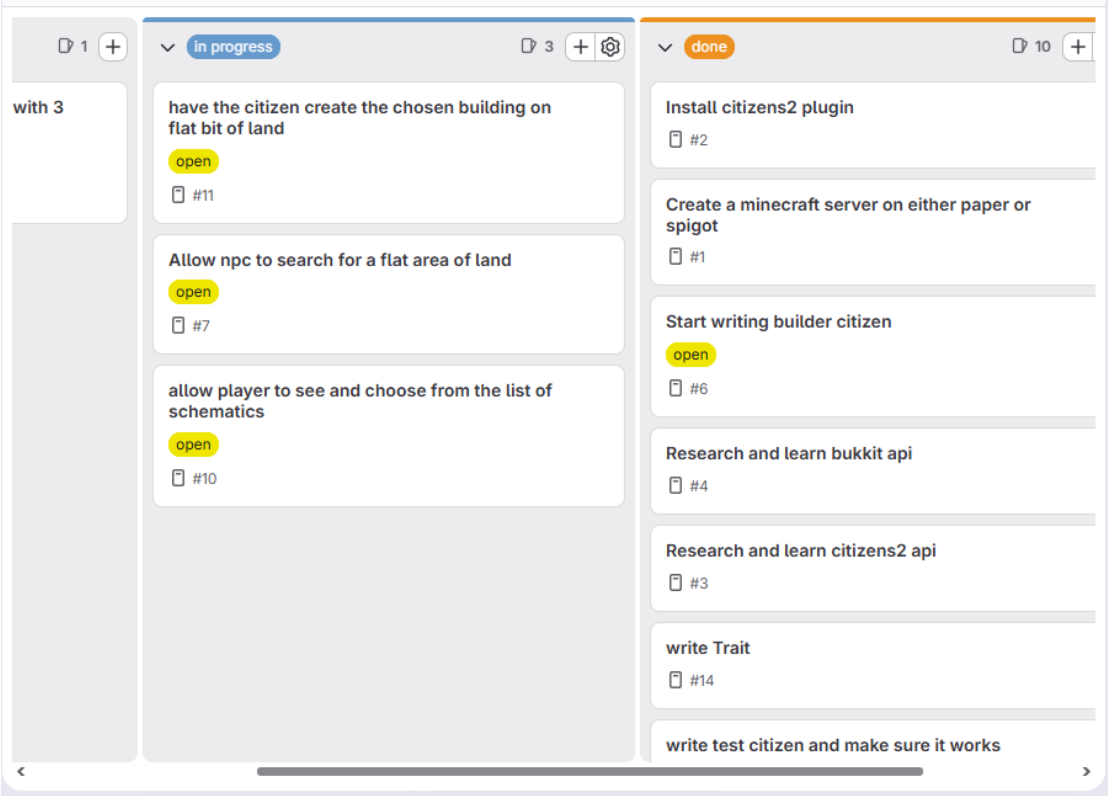


Figure 01: *Example of Gitlab's Kanban Board*

1.3.2. Planning and Task Management

Planning and Task Management were handled using Gitlab's Kanban issue board. The project was broken down into smaller tasks so that each part of the plugin could be worked on separately. This made the project easier to manage because the system included server different components, such as server setup, NPC creation, NPC traits, flat land search, schematic files, and block placement.

Each issue represented a task or problem that needed to be completed. Some examples included setting up the Paper server, testing if Citizens2 could create an NPC, trait behaviour, etc. Some issues that came up while testing each function were also treated as tasks however, not added as a separate issue. For example, during the flat land search implementation the NPC was moving to invalid land, this task then could not be completed until the issue was resolved.

While the Kanban board was very good to help prioritise tasks by making the state of the project visible, it was not always updated and was used more as a checklist than a fully maintained issue tracker. It did make it easier to focus on one task at a time and have some organization by having a clear list of remaining work rather than trying to complete the whole project at once.

1.3.3. Version Control

Gitlab was used to store the project repository and keep the plugin code in one central location. This meant that the Java classes, schematic files, and project materials could be kept

together with the issue board. Gitlab was also useful because it provided a backup of the project and allowed the code to be accessed if needed [12].

Version control was used more during the earlier stages of the project. At the beginning, commits were made when setting up the projects structure, testing the server start up, and creating the early plugin features. This helped with the initial development of the plugin and showed progress from a basic setup towards the working plugin.

However, as the project became more complicated, commits became less frequent as it was the last thing to think about and hasn't become a habit. Later development required a lot of testing in Minecraft, especially working on NPC movement, flat land searching, schematic file reading, and directional blocks. Due to focusing on getting it completed progress was not committed as regularly as it should have been if at all.

This was a limitation of the development process. More commits would have made it easier to track when bugs were introduced and would've made it more clear how it evolved. If the project were restarted this would become a priority and made into a habit pf doing.

1.4. Status of work

This section summarises the current state of the project. It explains what parts of the project brief have been completed, what the plugin is currently able to do, and what parts remain unfished. It also identifies issues in the current version and planned future work which could improve hr NPC builder plugin.

1.4.1. Completed Work

The completed work includes setting up the Minecraft server environment. Installing Paper, and adding the Citizens2 plugin. The server was tested to make sure everything ran smoothly and could be joined through Minecraft and that custom plugins could load.

A custom Java plugin was created using the Bukkit and Paper API, The plugin can start with the server, register event listeners, and respond to player chat messages. This allows the player to interact with the system using simple commands such as listing the schematics, selecting a schematic, and triggering the build process.

The plugin can create or reuse an NPC suing the Citizens2 API. A custom builder trait was also created and attached to the NPC. This trait stores the selected schematic and controls the main builder behaviour, including the flat land search and building the schematic

The flat land search has also been implemented. The search checks possible locations in Minecraft and validates if the area is suitable for building. The validation checks that the area is flat, large enough for the building, and doesn't contain blocks such as water or lava.

The schematic files were also implemented. The schematics store buildings using relative coordinates and block materials. Multiple schematics can be stored in the plugins schematic folder, and the plugin can calculate the require width and length of the building rather then using a fixed area.

The build command has also been implemented. When a flat area has been found the player can type build, and the plugin reads the selected schematic file and places the building into

Minecraft. This completes the workflow of selecting a schematic, finding flat land, and building a structure.

1.4.2. Current Capabilities

The current version of the plugin can complete the main problems of the project. The player can type schematics in the chat to view the available schematic files, then type the schematic name such as building_1, to select it. Once a schematic has been selected the plugin calculates the width and length of the building to find a flat area.

The NPC can move through the world using the Citizens2 navigation. When a valid area is found, the NPC moves towards that location, and the plugin stores the build coordinates. The player can then type build to place the selected schematic at the location.

The plugin can read text based schematic files that use coordinates and block materials. This allows buildings to be changed or added without changingthe Java code. This allows multiple building sizes because the required size is calculated from the schematic file.

The flat land search can validate whether a location is suitable by checking the size of the building. It checks that the terrain is level and rejects unsuitable blocks such as water and lava. This prevents the building from being placed on uneven or unsafe terrain.

1.4.3. Unfinished

The original idea was for the NPC to place the blocks for the building one by one with a real time animation. Now, the plugin places the whole building once the player types build rather then having the NPC place each block individually.

Another unfinished feature is terrain flattening. The current system searches for naturally flat land but does not modify the terrain if no suitable area is found. For large schematics, if there is no perfectly flat area within the search radius it fails to find a location.

Directional blocks are also still limited. Certain blocks such as stairs and doors, require extra block data to state their direction and placement. Right now, directional blocks work for the most part but look a bit silly. Some materials such as glass require more research on how to have the direction facing the correct way.

The user commands could also be improved. Currently players need to know the schematic file name and type it correctly. A future version would have clearer commands, better feedback, and a more user friendly schematic list.

1.4.4. Known Issues

There are several known issues in the current version of the project. I=One issue is that the flat land search struggles to find suitable land for large schematics. This is because the search requires the land to be perfectly flat having the tolerance set to zero. With bigger schematics, its less likely that a naturally generated Minecraft world will contain a perfectly flat area within the search radius.

Another issue is performance with large searches. Increasing the search radius gives the plugin more possible locations to check, but it also increase the amount of world data that needs to be read. If the radius is too large, the plugin can cause noticeable server lag because it has to check many block locations and ground heights.

Directional blocks are another issue. Blocks such as stairs and glass panes require more block data to control their direction and placement. The current schematics can place basic block materials, but directional blocks need more improvement to make roofs, doors, and windows look nice and consistent.

Finally, the commands require the player to spell the schematic name correctly. If the player misspells a schematic name or if the player tries to build a schematic before its returned a location it doesn't behave correctly. Better error messages returned to the player would make for a more enjoyable experience.

1.4.5. Planned Future Work

Any future work on this project would focus a lot on reliability and usability of the NPC plugin. The main improvement would be animated building. Instead of the whole building being placed down all at once the NPC would place each block individually in real time. This would make it look like the NPC builder is actually a proper builder.

Another planned improvement is better directional blocks and just having more complex schematics. Right now, the buildings being created are pretty basic and to have better placed stairs, doors, windows, trapdoors, or signs would add nice detail to the buildings but would require extra block data to control the direction and placement.

The flat land search could also be improved. A future version could either use a more efficient search algorithm or include terrain flattening where the NPC creates the correct bit of land before building. This would make it a lot better to place larger structures while dealing with the terrain.

More schematic s could also be made =meaning more buildings to choose from. The player would also be given a clearer way to choose between each schematic. For example, a menu based selection system would be nice to have.

Finally, the plugin could include better feedback and error handling. If no suitable land is found for a schematic or if a player types build before a location has been found, the plugin could give clearer messages explaining what happened and what the player should do next.

2. Design

2.1. Design Overview

This section gives an overview of the main design decisions made for the NPC builder plugin. The main stages are selecting a schematic, searching for suitable flat land, and building the selected structure The design focuses on separating the system into clear parts so that each class has a specific responsibility and is easier to understand, test, and improve.

The main design decision was to make the selected schematic control the size of the land required. This means the plugin can support multiple building sizes instead of only searching for one fixed area.

2.1.1. Design Goals

The main design goal was to create a working NPC builder plugin that would be controlled by simple player commands. The player should be able to choose a schematic, have the NPC search for suitable flat land, and then build the selected building.

Another goal was to keep the code organized by separating the main responsibilities. The plugin class handles server setup, NPC creation, and chat commands. The builder trait handles the selected schematic and build behaviour. The flat land class handles the search and validation of locations.

Th system was also designed to support different building sizes. Instead of always searching for a fixed area, the plugin calculates the required width and length from the selected schematic file.

The final goal was to make the schematic files easy to expand. New buildings can be added as text files without needing to rewrite code.

2.1.2. Scope

The scope of the design is focused on the main work flow of the NPC builder plugin. This includes allowing the player to list available schematics, select a schematic, have the NPC search for flat land, then build the selected structure.

The design includes the use of Bukkit and Paper for server integration and Citizens2 for creating and controlling these NPC. It also includes a text based schematic system where is building is stored using relative coordinates and block materials.

Some features are outside the current scope. The NPC doesn't build each block one at a time, and the plugin does not flatten or build land if no suitable spot is found. These are planned as future improvements.

2.2. System Architecture

This section explains how the main parts of the NPC builder plugin are structured and how they work together. The architecture separates the system into different responsibilities including player input, NPC behaviour, flat land search, and schematic building. This makes the system easier to understand and extend later.

The main components are the plugin class, the builder trait, the flat land search class, and the schematic files. The plugin class acts as the entry point, while the builder trait controls the NPC's behaviour. The flat land class handles the search for valid land, and the schematic files store the building data used during construction

2.2.1. Architecture Diagram

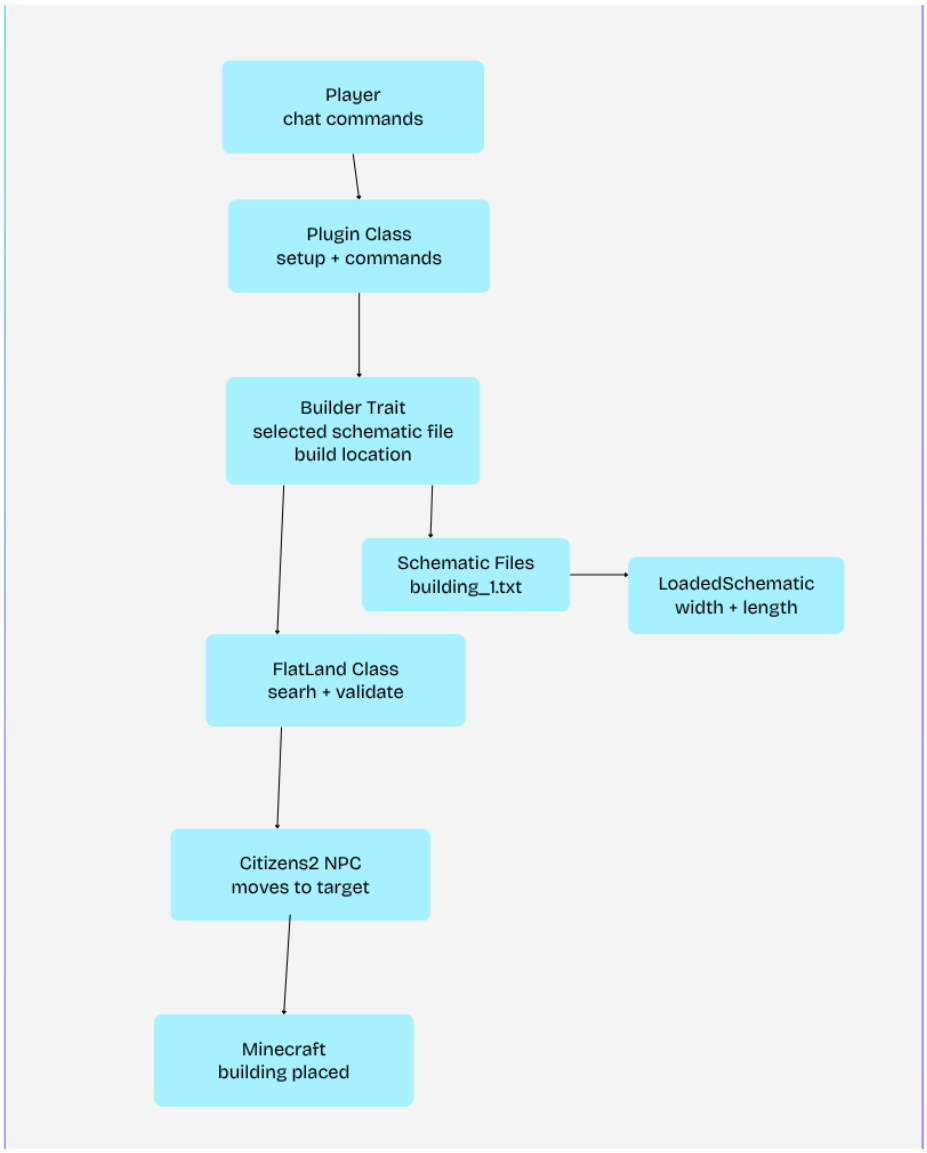


Figure 02: *System Architecture of the NPC builder plugin made using Canva[]*

2.2.2. Component Responsibilities

Each component in the plugin has specific responsibilities. This helps keep the code organised and makes the system easier to understand and improve.

The Plugin class is the main entry point of the plugin. It is responsible for setting up the plugin when the server starts, registering the chat event listener, creating for reusing the NPC, and attaching the builder trait to the NPC. The chat event listener handles player chat messages such as schematics, schematic names like building_1, and build.

The Builder trait class stores the main builder behaviour for the NPC. It keeps track of the selected schematic file and the build location found by the search. It also starts the flat land search and runs the build process after the player types build.

The FlatLand class is responsible for finding a build location. It generates possible locations, checks if the area is flat, makes sure the area is large enough for the schematic, and rejects unsuitable blocks such as water and lava. When a valid location is found, it sends that location back to the builder trait.

The schematic files store the building data. Each schematic is a text file containing relative coordinates and block materials. This allows buildings to be changed or added without editing the code.

Citizens2 is responsible for movement in Minecraft. The plugin chooses the target location, but Citizens2 handles the navigation that moves the NPC towards the selected build site.

2.2.3. Data Flow

The data flow begins when the player types a chat command. If the command is schematics, the plugin returns the available schematic files. If the command is a schematic name, the plugin class passes this to the Builder trait.

The Builder trait stores the selected schematic name and loads the matching schematic file. The schematic is used to calculate the width and length of the flat land needed by reading the minimum and maximum x and z offsets. These dimensions are then passed into the FlatLand class so it can search for a valid build area

After the location is found its stored in the Builder trait. The player can type build, which has the schematic be read and placed into the world using the saved location.

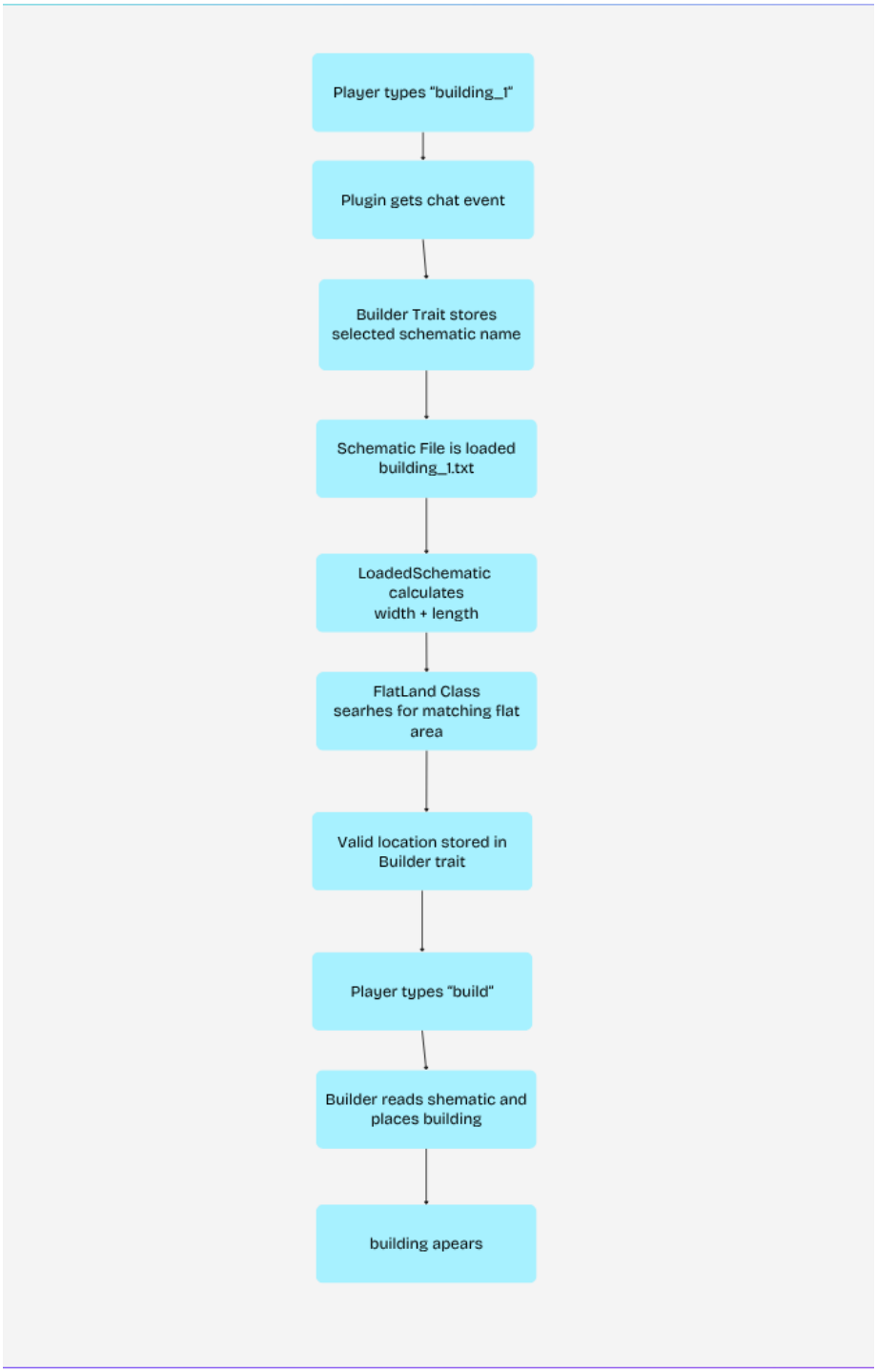


Figure 03: *Data Flow Diagram showing schematic selection to building placement*

2.2.4. State Management

The plugin stores a small amount of state so that the search and build stages can work separately. This is important because the player selects a schematic and starts the search, then later types build after a location has been found.

The main state is stored in the Builder trait. This includes the currently selected schematic, the flat land search object, and the saved build location. The selected schematic is stored so that the same file can be used during both the search and build staged. The build location is

stored after the FlatLand class finds a valid area allowing the plugin to place the building after the player types build.

The FlatLand class also stores temporary search state. This includes the queue of possible locations, the current target location, the repeating Bukkit task, and if the search is still active. The stat is cleaned or reset when a new search begins so that an old search does not interfere with a new schematic selection.

The separation of state helps keep the work flow clear. The Plugin class handles player commands, the builder trait stores the selected schematic and build position, and the FlatLand class manages the temporary state needed during the search process.

2.3. Schematics

This section explains how schematics are used as building blueprints in the plugin. A schematic is stored as a text file rather than being hardcoded. This makes it easier to add or edit buildings without changing the main plugin code.

The schematic system is responsible for two tasks. First, it tells the plugin which blocks need to be placed and where they should be placed. Second, it allows the plugin to calculate the width and length of flat land required before the NPC starts searching. This means the flat land search can change depending on the selected schematic. See ***Figure 04*** and ***Figure 05*** to see the structure of the schematic [13].

```
# floor
0 0 0 OAK_PLANKS
1 0 0 OAK_PLANKS
2 0 0 OAK_PLANKS
3 0 0 OAK_PLANKS
4 0 0 OAK_PLANKS
0 0 1 OAK_PLANKS
```

Figure 04: *The x, y , z BLOCK_MATERIAL Structure of a Schematic File*

```
0 4 0 OAK_STAIRS facing=east,half=bottom
0 4 1 OAK_STAIRS facing=east,half=bottom
0 4 2 OAK_STAIRS facing=east,half=bottom
0 4 3 OAK_STAIRS facing=east,half=bottom
0 4 4 OAK_STAIRS facing=east,half=bottom
```

Figure 05*: Example of the text based schematic format with more block data*

2.3.1. Schematic Implementation

the selected schematic is plugin's schematics folder after the player types its file name without .txt, such as building_1. The plugin reads the file line by line, ignoring blank lines and comments, then splits it into x, y, z coordinates and a Bukkit material name [15].

The schematic is used for searching and building. During the search, the plugin finds the minimum and maximum x and z values to calculate width and length for the land. During building, the coordinates are added to the saved build location to place the building.

2.3.2. Build Origin

The build origin is the world location used as the starting point for placing a schematic. Each x, y, and z value in the schematic is added to the origin to calculate the final block position..

The flat land search returns the corner of the valid build area as the origin because the schematics start with 0, 0, 0. This allows the same schematic to be placed anywhere in the world while lining up with the flat area.

2.3.3. Directional Blocks

Some Minecraft blocks require extra block data as well as material name. For example, doors need a direction and an upper and lower half. The schematic can include this extra DATA SO Bukkit BlockData can plae blocks more accurately then setType() alone. Directional blocks were a challenge because roofs, doors, and windows need more block data to show correctly.

2.4. Search

This section explains how the search finds a flat area that matches the selected schematics required width and length. Locations are checked and only allowed if the area is level and doesn't contain unsuitable blocks like water or lava.

2.4.1. Valid Build Site

A valid build site is a flat area that matches the selected schematics width and length. It is calculated from the minimum and maximum x and z values in the schematic file. The tolerance is currently zero, so every ground block must be at the same height. Water and lava are also rejected.

2.4.2. Search Space and Parameters

The search is controlled by parameters such as radius, step, width, length, and tolerance. A larger radius checks more land but can slow the server, while a larger step checks fewer locations and could skip valid flat areas.

2.4.3. Grid Search

The plugin uses a grid-based search that expands outward from the search origin. Locations are checked closest first and are only given to the NPC if it validates. Citizens2 then handles the movement to the selected target.

2.4.4. Validation

Validation checks whether a location is suitable before the NPC moves to it. The plugin checks the full schematic, compares all ground heights against the center height, and rejects the candidate if any block is outside the allowed tolerance. Since the tolerance is zero, the whole footprint must be level.

2.4.5. Failure

The search fails if no valid location is found within the search radius. This is more likely to happen with larger schematics because they need a bigger flat area. If this happens, no build location is stored, so the player must try a different area, use a smaller schematic, or increase the search radius.

2.5. NPC Behaviour

This section explains how the NPC behaves during the search and build process. Citizens2 controls the NPC, while the plugin decides when it should search, where it should move, and when the selected schematic should be built.

2.5.1. Citizen Traits

Citizens2 traits attach custom behaviour to an NPC. In this project, the custom builder trait stores the selected schematic, the current search object, and the saved build location. This keeps NPC-specific behaviour separate from the main plugin setup code.

2.5.2. Navigation

The NPC uses Citizens2 navigation to move towards the selected build site. The plugin chooses the target location after validating the land, while Citizens2 handles the pathfinding and movement.

2.5.3. Scheduler

The Bukkit scheduler runs repeated checks while the NPC is moving. This lets the plugin check whether the NPC has reached the target without running everything at once. The task uses Minecraft ticks, where 1L is one tick and 10L is half a second.

2.5.4. Commands

The player controls the NPC through chat commands. schematics lists available files, a schematic name such as building_1 starts the search, and build places the selected structure after a valid location has been found. This keeps searching and building separate.

3. Implementation

3.1. Implementation Overview

The plugin was implemented in Java using Paper, Bukkit and Citizens2. The main implementation connects player chat commands, NPC creation, schematic loading, flat land searching, and block placement into one workflow. The code is split into separate classes for plugin setup, NPC behaviour, schematic data, and land searching.

3.1.1. Development Environment and Setup

The project was developed in Java using IntelliJ IDEA, Maven, Paper, Bukkit, and Citizens2. The plugin was build into a .jar file and placed into the servers plugin folder. The Bukkit and Paper configuration was stored in plugin.yml [5], and Maven dependencies were managed through pom.xml [9], [10].

3.1.2. Project Structure

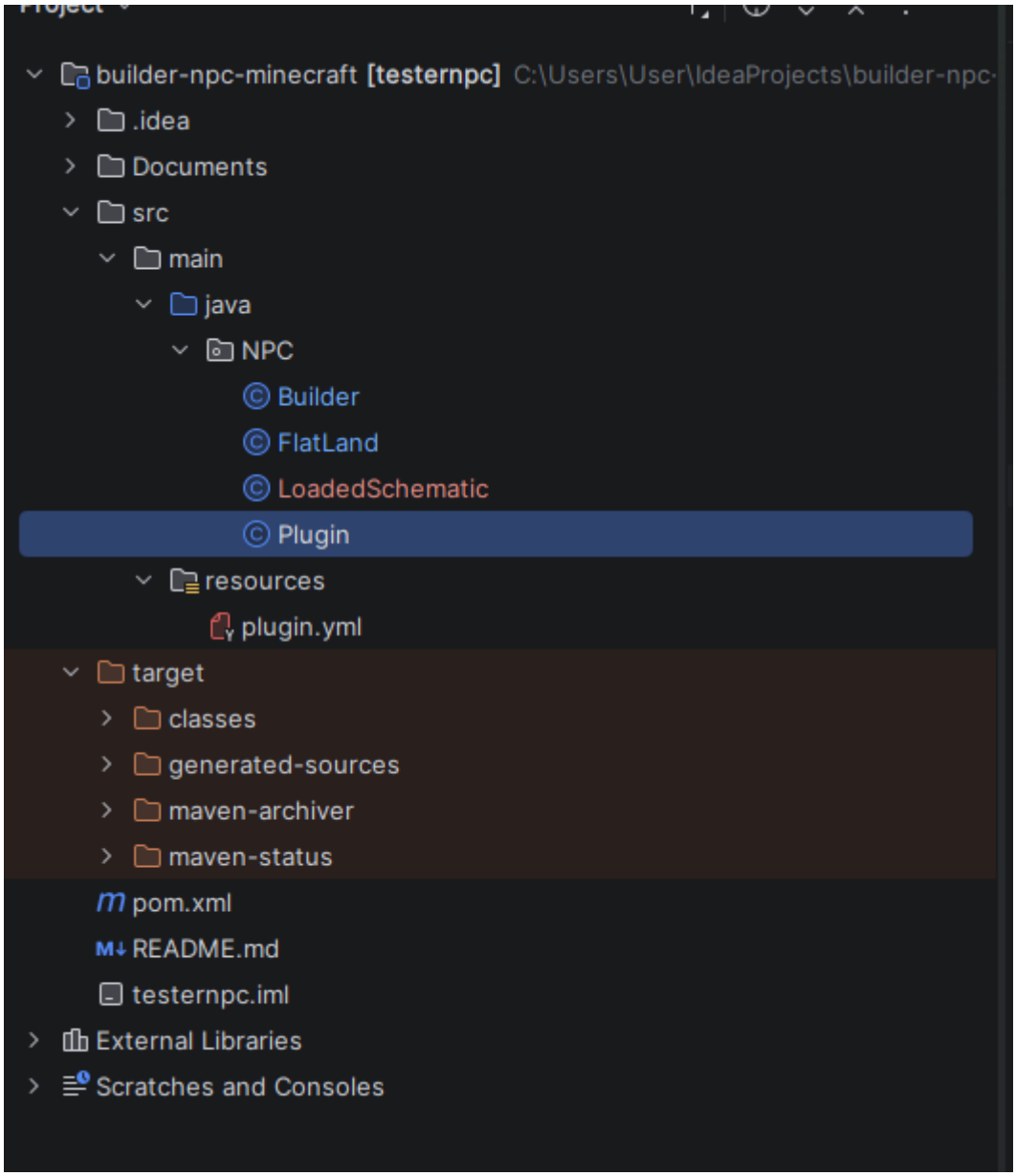


Figure 06: *Project structure in IntelliJ IDEA*

3.2. Plugin Entry and Event Handling

This section explains how the plugin starts, registers events, and responds to player chat commands.

3.2.1. Plugin Configuration

The plugin is configured using plugin.yml, which tells Paper the plugin name, version, main class, and dependencies. Citizens2 is included as a dependency because the plugin needs the Citizens API to create and control the NPC.

3.2.2. Plugin Lifecycle

When the server starts, the plugin's onEnable() method runs. It registers the chat event listener, registers the custom Citizens builder trait, and creates or reuses the NPC called Jeff.

If Jeff is not already spawned, the plugin spawns him in the world and attaches the builder trait.

3.2.3. Chat Event Handling

Player chat messages control the main plugin workflow. Typing schematics lists available schematic files, typing a schematic name such as building_1 selects it and starts the search, and typing build places the selected structure after a valid location has been found.

The chat event is scheduled back onto the main server before interacting with the world or NPC, because Bukkit world operations and Citizens navigation should not be run directly inside the asynchronous chat event [5].

3.3. Citizens NPC Integration

Citizens2 is used to create, store, and control the builder NPC. The plugin decides what the NPC should do, while Citizens2 provides the NPC object, navigation, and trait system[6].

3.3.1. NPC Creation and Reuse

When the plugin starts, it checks the Citizens NPC registry for an existing NPC called Jeff. If one exists, it is reused otherwise a new NPC is created. This prevents duplicate NPCs from being created every time the server restarts.



Figure 07: *Builder NPC Jeff spawned in the Minecraft world.*

3.3.2. Navigation Setup

Citizens2 navigation moves the NPC towards the selected build site. The plugin gives the navigator a target location after the land search finds a valid area. Old navigation targets are cancelled before a new search starts so the NPC does not continue moving towards an outdated location.

3.3.3. Trait Attachment

The custom builder trait stores the selected schematic, current search object, and saved build location. It is registered when the plugin starts and attached to the NPC using getOrAddTrait, making sure the NPC always has the required builder behaviour.

3.4. Schematic Loading

This section explains how text-based schematic files are loaded and used by the plugin.

3.4.1. File Location

The schematic files are stored inside the plugin's data folder in a folder called schematics. For example, a schematic file may be stored as:

plugins/Plugin/schematics/building_1.txt

The player selects a schematic by typing its name without the .txt extension, such as building_1. The plugin then adds .txt in the code and looks for the matching file in the schematics folder [15], [16].

3.4.2. Schematic Format

Each schematic is written as a text file. Each valid line represents one block that should be placed in the Minecraft world. The general format is:

x y z MATERIAL

The x, y, and z values are relative coordinates. They describe where the block should be placed compared to the final build origin. The MATERIAL value is the Bukkit material name, such as OAK_PLANKS.

The schematic files can also contain comments and blank lines. Comment lines start with # and are ignored by the plugin. This makes the schematic files easier to organise into sections such as floor, walls, windows, doors, and roof.

3.4.3. Directional Blocks

Directional blocks require extra block data. Doors need an upper and lower half while stairs need a facing direction. The plugin can read this data and use Bukkit BlockData [5] to place these blocks more accurately, although this area still needs to be improved.

3.5. Flat Land Search Implementation

The FlatLand class searches for terrain matching the selected schematic's required width and length.

3.5.1. Candidate

A candidate is a possible location that could be used as a build site. The plugin creates candidate locations around the search origin and placed one block above ground so the NPC doesn't stand inside the terrain.

.

3.5.2. Algorithm Used

The search uses a grid-based search that expands outward from the starting location. It starts near the centre point and then checks locations further away within the search radius, meaning closer locations are checked first.

The search is controlled by radius and step. The radius controls how far the plugin searches, while the step controls how many blocks it moves between candidate positions. A grid search was chosen because the goal is to find a valid build site, not to calculate the NPC's path. Citizens2 handles the movement after a valid target is found.

3.5.3. Validation

Validation checks whether a candidate is suitable for the selected schematic. The plugin checks the full footprint using the schematic's width and length.

For each block in the footprint, the ground height is compared with the centre height. Since the tolerance is zero, the whole area must be perfectly flat. Water and lava are also rejected so the building is not placed in unsuitable terrain.

3.5.4. Return Build location

When a valid candidate is found, the NPC is sent to that target using Citizens2 navigation. Once the NPC reaches it, the plugin calculates the build location.

The build location is returned as the corner of the valid flat area rather than the centre. This is needed because schematic coordinates start from zero, so using the corner lets the building line up correctly with the validated area.

3.6. Scheduling

Scheduling is used because the NPC does not instantly reach its target. The plugin needs to repeatedly check whether the NPC has arrived before storing the build coordinates.

3.6.1. Bukkit Scheduler

The Bukkit scheduler runs a repeating task during the search process. Once the NPC is given a target, the task checks the NPC's current position and decides whether it has reached the location.

The task is stored so it can be cancelled when the search ends or when a new search begins. This prevents old search tasks from continuing in the background.

3.6.2. Tick timing

Minecraft servers run using ticks, with 20 ticks per second. The search task can repeat every 10L ticks, which is about half a second.

This timing is frequent enough to detect when the NPC reaches the target, but not so frequent that it checks every tick unnecessarily.

3.7. Building Implementation

This section explains how the selected schematic is placed into the Minecraft world. The build process happens after the player has selected a schematic and the flat-land search has returned a valid build location.

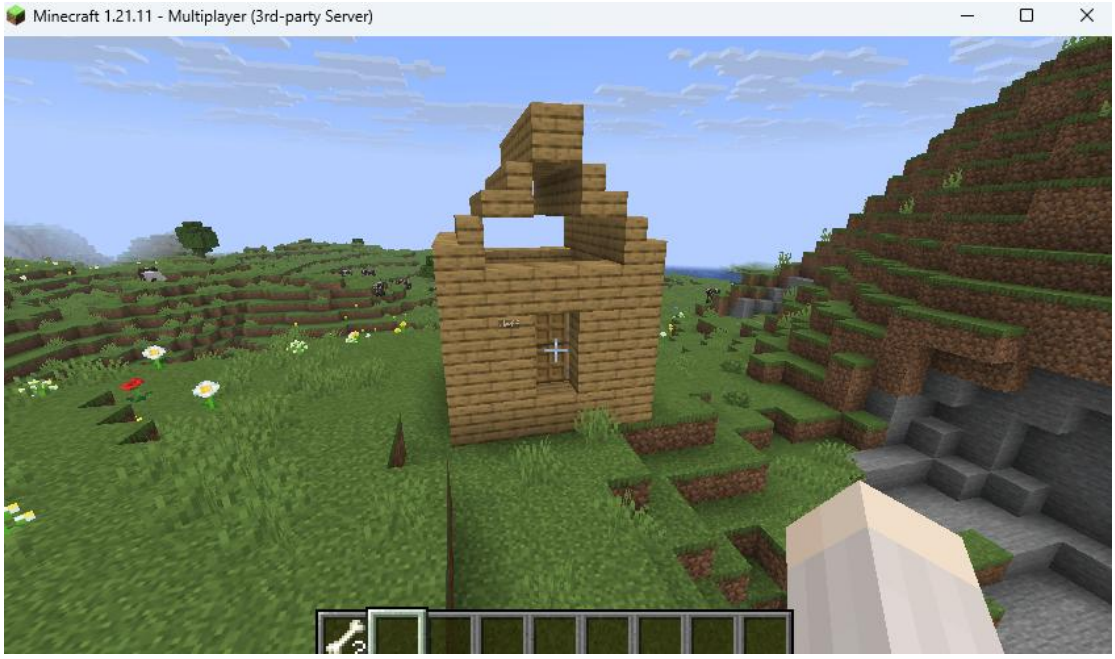


Figure 08: *Completed schematic built by the NPC builder plugin*

3.7.1. Build Trigger

The build process starts when the player types build. This command is separate from schematic selection because the plugin should not place blocks before a valid location has been found.

Before building, the plugin checks that a build location exists. If there is no saved location, the build process should not continue.

3.7.2. Block Placement

When build is triggered, the plugin reads the selected schematic file. Each relative coordinate is added to the saved build location to calculate the final world position:

worldX = buildLocationX + schematicX
worldY = buildLocationY + schematicY
worldZ = buildLocationZ + schematicZ

Simple blocks are placed using the Bukkit material name. Directional blocks, such as stairs and doors, can use Bukkit BlockData so extra properties like direction or door half can be applied.

3.8. Problems Encountered

This section describes the main technical problems found during implementation and testing.

3.8.1. NPC Confused

One problem encountered was that the NPC sometimes appeared confused during the land search. This happened when the NPC was sent towards locations before the area had been confirmed as suitable for building. As a result, the NPC could move towards invalid locations or appear to change direction unexpectedly.

To fix this, the search was changed so that a candidate location is validated before the NPC is given it as a target. The plugin checks the full required variables first, including the flatness of the area and whether there are unsuitable blocks such as water or lava. After this validation succeeds is the NPC sent towards that location.

Another related issue was that the NPC could sometimes be in an awkward position after a building had been placed, like near or inside the structure. This could affect Citizens2 navigation when starting a new search. To improve this, navigation is cancelled before a new search begins so that the NPC is not still trying to move towards an old target.

3.8.2. Search/Build Command Coupling

Another problem was that the search and build stages were initially too closely connected. Typing a command could trigger the search and build process together. This caused issues because the plugin could try to build before the NPC had actually found suitable land.

This was fixed by separating the commands. Typing a schematic name now starts the land search and stores the build location when a valid area is found. The build command is handled separately and only attempts to place the structure after the search has completed.

This separation made the system more reliable because the selected schematic, land search, and building process happen in a clear order. It also made the user interaction easier to understand, first choose a schematic, then wait for a valid location, then type build.

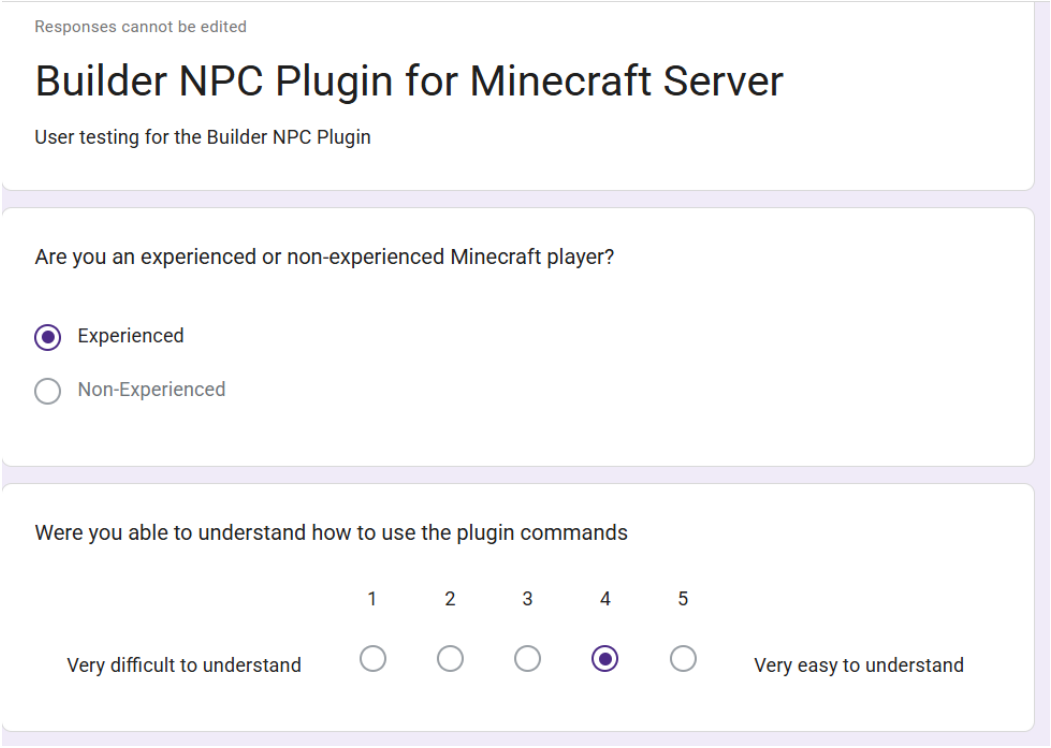
4. Testing

4.1. Functional Testing

Test ID	Requrement	Test	Expected	Actual Result	Pass/Fail
FT1	FR1, FR2	Start the Paper server with Citizens2 and the custom plugin installed.	Server starts successfully and loads both Citizens2 and the Builder NPC plugin	Server runs successfully	P
FT2	FR3	Join the server after startup	NPC Jeff is created or reused and appears in the world.	Jeff is in the server	P
FT3	FR5	Type schematics in chat.	Plugin returns a list of available schematic files.	returns schematic list	P
FT4	FR6	Type a valid schematic name	Plugin selects the matching schematic and starts the flat land search.	Plugin selects the matching schematic and starts the flat land search.	P
FT5	FR7, FR8	Select building_1 and allow the plugin to read the file.	Plugin reads the schematic and calculates the required width and length.	Reads and calculates width and length	P
FT6	FR8	Select a larger schematic	Plugin uses the larger schematic w + L when searching for land	It tries but fails have to type it multiple times for it to work	F
FT7	FR9, FR11	Wait after selecting a schematic	NPC moves towards suitable flat land and the plugin returns the build coordinates	Does for building 1 takes a while for the bigger ones	P/F????
FT8	FR10	After coordinates are returned, type build	Plugin places the selected schematic at the stored build location	Does for building 1 takes a while for the bigger ones	P/F????
FT9	FR12	Build one schematic, then select another valid schematic	Plugin starts a new search for the newly selected schematic	It does start a new search	P

4.2. User Testing

4.2.1. Experienced



Responses cannot be edited

Builder NPC Plugin for Minecraft Server

User testing for the Builder NPC Plugin

Are you an experienced or non-experienced Minecraft player?

- (●) Experienced
- ( ) Non-Experienced

Were you able to understand how to use the plugin commands

	1	2	3	4	5	
Very difficult to understand	○	○	○	●	○	Very easy to understand

Was it clear how to choose a schematic?

	1	2	3	4	5	
Very difficult to choose	○	○	○	◉	○	Very easy to choose

Did the NPC successfully find a flat area based on thew chosen schematic?

- ◉ Yes
- ○ No
- ○ Other: .....

Did the building appear in the correct location?

- ◉ Yes
- ○ No
- ○ Other: .....

Did anything confuse you while using the plugin?

At times the NPC would be on a flat land that could hold the building but chose not to build

Did anything go wrong or not work?

No, everything seemed to work properly

How easy was it for you to use the plugin overall?

It was fairly simple

What improvements would you like to see?

I would have liked to see what the buildings looked like before deciding on 1,2,3,4

Would you use the plugin again?

- (●) Yes
- ( ) No
- ( ) Other:

4.2.2. Non-Experienced

Responses cannot be edited

Builder NPC Plugin for Minecraft Server

User testing for the Builder NPC Plugin

Are you an experienced or non-experienced Minecraft player?

- ( ) Experienced
- (●) Non-Experienced

Were you able to understand how to use the plugin commands

	1	2	3	4	5	
Very difficult to understand	( )	( )	( )	( )	(●)	Very easy to understand

Was it clear how to choose a schematic?

	1	2	3	4	5	
Very difficult to choose	○	○	◉	○	○	Very easy to choose

Did the NPC successfully find a flat area based on thew chosen schematic?

- ◉ Yes
- ○ No
- ○ Other: .....

Did the building appear in the correct location?

- ◉ Yes
- ○ No
- ○ Other: .....

Did anything confuse you while using the plugin?

I didn't know what the buildings were, or what the houses looked like

Did anything go wrong or not work?

It all worked

How easy was it for you to use the plugin overall?

It was easy, just type what you wanted Jeff to do

What improvements would you like to see?

A menu saying what the commands were

Would you use the plugin again?

- (●) Yes
- ( ) No
- ( ) Other:

5. Critical Evaluation

5.1. Strengths

5.1.1. Technical Strengths

One technical strength of the project is that the system is split into separate classes with clear responsibilities. The plugin class handles server setup and chat events, the builder trait handles NPC behaviour, and the flat land class handles the search and validation. This separation makes the code easier to understand and improve.

Another strength is that the land search is linked to the selected schematic. Instead of always searching for a fixed size, the plugin calculates the required width and length from the schematic file. This makes the system more flexible because different buildings can require different amounts of flat land.

The text based schematic format is also a strength. It allows buildings to be added or edited without changing the Java code. This makes the building system easier to extend and demonstrates a clear separation between building data and plugin logic.

The project also successfully combines several technologies. It uses Bukkit and Paper for server interaction, Citizens2 for NPC creation and navigation, and Java file reading to load schematic data. Bringing these systems together into one workflow was one of the main technical achievements of the project.

5.1.2. Usability Strengths

A usability strength of the project is that the command flow is simple. The player can type schematics to list available buildings, type a schematic name to start the search, and type build after a valid location has been found. This makes the system fairly easy to understand.

Another strength is that the player does not need to manually choose exact coordinates for the building. The NPC searches for suitable land automatically which reduces the amount of work required from the player.

5.1.3. Process Strengths

A process strength was the use of an iterative development approach. Testing features directly inside Minecraft made it easier to identify problems that would not have been obvious from the code alone, such as NPC movement issues, unsuitable land selection, and directional block problems.

The Kanban-style task board also helped organise the project into smaller tasks. This made the project easier to manage because large features such as NPC creation, land searching, and schematic building could be developed in smaller stages.

Another strength was that the project scope was adjusted when needed. More complex features, such as animated block-by-block building and terrain flattening, were left as future work so that the core system could be completed first. This helped keep the project focused on delivering a working NPC builder workflow.

5.2. Weaknesses

5.2.1. Technical Limitations

One technical limitation is that the NPC does not yet build structures block by block in real time. It would be nice to see an animation of it rather then the building just spawning in. The current version, the schematic is placed after the build command is used, rather than being built gradually by the NPC.

Another limitation is that the land search depends on naturally flat terrain. The plugin does not currently flatten land before building. This means that larger schematics can be difficult to place because they need a larger area of perfectly flat land.

The search can also become expensive if the radius is too large. The plugin checks terrain positions and ground heights, and increasing the radius means more locations must be checked. This can cause performance issues if the search area becomes too large.

5.2.2. Reliability Limitations

A reliability issue is that the search may fail if no suitable flat land is found within the search radius. This is especially likely when the schematic is large and the tolerance is set to zero. If no valid land is found, the building cannot be placed until the player tries again from a different area or uses a different schematic.

Another reliability limitation is that Citizens2 navigation can behave unpredictably if the NPC is in an awkward position, such as inside a building or on top of a structure after building. This can affect the NPC's ability to move to a new search target.

5.2.3. Usability Limitations

A usability limitation is that the player must type the schematic name correctly. If the name is misspelled, the plugin cannot load the correct file. Although the schematics command lists available files, the commands could still be made more clear.

The plugin could also provide better feedback. For example, if no flat land is found, the player should be told that the search failed and why. Better feedback would make the system easier to use and easier to debug.

5.3. Improvements if Restarting

5.3.1. Design Improvements

If restarting the project, I would design the schematic system earlier. At first, the project focused on a fixed 5x5 search, but later the system needed to support multiple schematic sizes. Designing the project around schematics from the beginning would have made the search and build system cleaner.

I would also design the command flow more clearly from the start. Separating schematic selection, searching, and building became important later in the project. A clearer command structure, such as /builder list, /builder select building_1, and /builder build, would make the system easier to use and test.

5.3.2. Implementation Improvements

If restarting the implementation, I would focus more on error handling and feedback messages. This would make it easier to understand when a schematic file is missing, when no valid land has been found, or when the player tries to build too early.

Another improvement would be to implement directional block support more carefully from the beginning. Stairs, doors, and other directional blocks are more complex than simple block types, so designing the schematic format around BlockData earlier would have reduced later problems.

5.3.3. Testing Improvements

If restarting the testing process, I would create a more formal test plan earlier. The project was tested frequently in game, but recording test results from the start would have made the final evaluation stronger.

I would also test each feature separately before combining them. For example, schematic parsing, width and length calculation, flat land validation, and block placement could each have been tested individually before testing the full NPC workflow.

More user testing could also improve the project. Testing with more players would provide better feedback on whether the command system is clear and whether the plugin is easy to understand.

5.4. Future Work

5.4.1. Schematics

Future work could expand the schematic system by adding more building types and supporting more complex buildings. The current text based format works for simple buildings, but larger or more detailed structures would benefit from a more advanced format.

The plugin could also include a better schematic selection system. For example, players could use a command to list, select, and confirm schematics more clearly. A menu system could also make the plugin easier to use.

5.4.2. Directional Blocks

Directional blocks should be improved in future versions. Blocks such as stairs, doors, trapdoors, and signs need extra block data to place correctly. Better support for this would allow buildings to have more detailed roofs, entrances, and decorative features.

Another useful improvement would be schematic rotation. This would allow the same building to face different directions without needing separate schematic files for each orientation.

5.4.3. Faster Search

The flat land search could be improved to make it faster and more reliable. One improvement would be making it so the plugin does not repeatedly check the same locations. Another option would be limiting the search to already loaded chunks to avoid server lag.

A future version could also include terrain flattening. Instead of only searching for naturally flat land, the NPC could prepare the ground before building. This would make it much easier to place larger schematics in normal Minecraft terrain.

5.5. Conclusion

5.5.1. Summary

The project successfully developed a Minecraft NPC builder plugin using Java, Paper, Bukkit, and Citizens2. The system can create or reuse an NPC, attach a custom builder trait, list available schematics, select a schematic, calculate the required flat land width and length, search for suitable terrain, and place the selected building into the world.

The project demonstrates how Bukkit world interaction and Citizens2 NPC behaviour can be combined to create an automated builder system. Although some advanced features I wanted to complete, the core of the project by selecting a schematic, finding land, and building a structure was implemented.

5.5.2. Final Reflection

Overall, this project was challenging because it involved learning several technologies and combining them into one working system. Many issues only became clear during in game testing, especially problems with NPC navigation, land validation, search performance, and directional blocks.

The project improved my understanding of Java plugin development, event driven programming, APIs, file parsing, and iterative testing. If I continued the project, I would focus on animated building, better schematic support, improved search performance, and clearer user commands. One of the main lessons I've learned from the project is that features that seem simple an become so much more difficult when tested in game. Searching for flat land not only depended on the algorithm, but also the terrain, the loation of the NP, server performance, and schematic size. This is what made iterative testing important throughout the project. I really enjoyed utilizing the iterative methodology as I feel like I've finally found what works best for me. I will definitely be using it again for all of my future projects.

I have also learned that it would be beneficial for me to turn Gitlab into a habit and remember to push my code consistently as well as utilize the Kanban board more often. I feel as though these were very helpful I was slacking at bit when it came to remembering to use this.

When I continue working on this project in the future, I would focus on animated building, better schematic support, improved search performance, and better user commands. If I were to give myself a mark on this assignment I truly don't know what I would give myself. Although my project has some flaws, I'm very proud of what I've accomplished despite the hardhsips I've faced this term.

6. References

[1] Mojang Studios, “What is Minecraft?”, Minecraft.net. Available: https://www.minecraft.net/en-us/article/what-minecraft [Accessed: 30-Apr-2026].

[2] PaperMC, “Paper Documentation,” PaperMC Docs. Available: https://docs.papermc.io/paper/ [Accessed: 30-Apr-2026].

[3] PaperMC, “Paper,” PaperMC. Available: https://papermc.io/software/paper [Accessed: 30-Apr-2026].

[4] SpigotMC, “Bukkit API Javadocs,” Spigot Hub. Available: https://hub.spigotmc.org/javadocs/bukkit/ [Accessed: 30-Apr-2026].

[5] SpigotMC, “Bukkit Class Documentation,” Spigot Hub Javadocs. Available: https://hub.spigotmc.org/javadocs/bukkit/org/bukkit/Bukkit.html [Accessed: 30-Apr-2026].

[6] Citizens, “Citizens Wiki,” Citizens Wiki. Available: https://wiki.citizensnpcs.co/Citizens_Wiki [Accessed: 30-Apr-2026].

[7] Citizens, “Citizens API,” Citizens Wiki. Available: https://wiki.citizensnpcs.co/API [Accessed: 30-Apr-2026].

[8] CitizensDev, “Citizens2 GitHub Repository,” GitHub. Available: https://github.com/CitizensDev/Citizens2 [Accessed: 30-Apr-2026].

[9] Oracle, “Java Documentation,” Oracle. Available: https://docs.oracle.com/en/java/ [Accessed: 30-Apr-2026].

[10] Apache Maven Project, “Maven Documentation,” Apache Maven. Available: https://maven.apache.org/guides/ [Accessed: 30-Apr-2026].

[11] GitLab, “Issue Boards,” GitLab Docs. Available: https://docs.gitlab.com/user/project/issue_board/ [Accessed: 30-Apr-2026].

[12] GitLab, “Issues,” GitLab Docs. Available: https://docs.gitlab.com/user/project/issues/ [Accessed: 30-Apr-2026].

[13] EngineHub, “Clipboard,” WorldEdit Documentation. Available: https://worldedit.enginehub.org/en/latest/usage/clipboard/ [Accessed: 30-Apr-2026].

[14] Minecraft Wiki, “Structure Block File Format,” Minecraft Wiki. Available: https://minecraft.fandom.com/wiki/Structure_Block_file_format [Accessed: 30-Apr-2026].

[15] SpigotMC, “Material,” Spigot API Javadocs. Available: https://hub.spigotmc.org/javadocs/bukkit/org/bukkit/Material.html [Accessed: 30-Apr-2026].

7. Appendices

7.1. Use of Third-Party Code and Libraries

Third-party tool or library	Purpose in the project
Minecraft	Used as the game environment where the NPC builder plugin was tested.
Paper	Used as the Minecraft server software for running the custom plugin.
Bukkit API	Used to create the plugin, listen for chat events, access world locations, check block types, schedule tasks, and place blocks.
Citizens2	Used to create, reuse, and control the builder NPC called Jeff.
Citizens2 API	Used to attach the custom builder trait and control NPC behaviour and navigation.
Java	Main programming language used to develop the plugin.
Maven	Used to manage the project configuration and dependencies.
IntelliJ IDEA	Development environment used to write and build the Java plugin.
GitLab	Used for repository storage, issue tracking, and the Kanban board.
Canva	Used to create report diagrams, such as the architecture and data flow diagrams.

No third party source code was copied directly into the submitted plugin. Third party APIs and tools were used through their documented functionality.